# Proflog: A Tableau-Based Logic Programming Kernel in miniKanren

ANONYMOUS AUTHOR(S)

We report on an implementation of Melvin Fitting's Proflog, a logic programming language based on the tableau proof procedure. The system extends a first-order tableau prover with equality, written in Clojure's `core.logic`, with Fitting's Procedure Call rule. Formulae thereby acquire a programmatic character: evaluating a program is a matter of closing the relevant tableau. Following alphaLeanTAP, successful evaluation also produces an explicit proof object. We describe the proof kernel, the surface-to-kernel translation, and worked executions of Fitting's reference programs.

CCS Concepts: • **Theory of computation** → **Logic and verification**; *Automated reasoning*; • **Software and its engineering** → *Constraint and logic languages.*

Additional Key Words and Phrases: miniKanren, logic programming, tableaux, Proflog, proof search

**ACM Reference Format:**
Anonymous Author(s). 2026. Proflog: A Tableau-Based Logic Programming Kernel in miniKanren. In *Proceedings of The 2026 miniKanren and Relational Programming Workshop (miniKanren 2026).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 INTRODUCTION

Fitting proposed Proflog as an exploration in alternative bases for logic programming [3]. Prolog operationalizes Horn clauses by resolution. Proflog instead manipulates formulae of first-order logic with equality by a tableau proof procedure, augmented with a mechanism for recursive procedure calls. The core tableau procedure is standard: branch closure, alpha/beta decomposition of connectives, gamma/delta treatment of quantifiers, and equality over a weak Herbrand setting. The programming step is the Procedure Call rule. During an attempt to close a tableau, a saved literal whose predicate is defined by the program may open a subsidiary tableau for the corresponding clause body.

This changes the usual programming/proving boundary. In Proflog, a program is not a collection of Horn clauses with procedural negation as failure. It is a finite collection of first-order definitions, at most one per relation symbol, whose queries are interpreted by two proof searches: a query succeeds when the negated query closes, fails when the query itself closes, and is unresolved when neither direction closes within the admitted search. This mirrors Fitting's supervaluation semantics and makes some operational warnings visible, especially the warning that factoring a formula into an auxiliary relation may change the program.

The implementation reported here is written in Clojure and `core.logic` in the miniKanren tradition [2, 5]. Its immediate prover lineage is alphaLeanTAP, a relational tableau prover for first-order classical logic [4], and a Clojure/core.logic.nominal implementation reference by Amin [1]. Proflog extends that lineage in the direction Fitting sketched: first-order proof search becomes a program evaluator, and program evaluation returns proof objects whose step tags record the tableau work performed.

## 2 IMPLEMENTATION

### 2.1 Proof Kernel

The kernel is a relation over proof states. Its central entry point is `prove-stateo`; public calls use `kernel/prove` for direct formulae and `kernel/prove-program` for program-bearing formulae. A branch state contains the focused formula, pending formulae, saved literals, the lexical environment

for bound variables, gamma-introduced proof variables, delta parameters, an explicit equality substitution, delayed disequalities, the compiled program, fuel, and the proof term being constructed.

The equality design is deliberately explicit. Positive equality extends a free-constructor substitution; negative equality is retained in a symbolic disequality store until later bindings force closure. This keeps equality, disequality, and procedure-call interaction visible in proof objects. A small closure proof, for example, has the shape:

```
(conj (savefml (close)))
```

An equality/disequality proof exposes the order of work:

```
(conj (neq-store (eq-step (eq-bind) (neq-close))))
```

Procedure calls add `pos-call` and `neg-call` proof steps. A positive call closes the relation body. A negative call closes the normalized negation of that body. Both calls run under the same kernel relation rather than through host-language evaluation of the source program.

### 2.2 Surface Syntax and Descent

The implemented frontend is a Clojure-readable prefix DSL that preserves Fitting's clause organization. The intended handwritten notation is:

```
p(x) :- x = a.

only-zero(x) := forall y. (x != y or y = zero).
zero-only(x) :- only-zero(x).
```

The frontend form uses `|-` for a real Proflog clause and `:=` for a frontend-only abbreviation:

```
(def peano-language
  (pf/language
    (constants zero)
    (functions (s 1))
    (relations (zero-only 1))))

(def zero-only-program
  (pf/proflog peano-language
    (:= (only-zero x)
      (forall [y]
        (or (!= x y)
            (= y zero))))
    (|- (zero-only x)
        (only-zero x))))
```

After macro expansion and compilation, the kernel sees tagged formula data. The simple query `p(a)` descends schematically to:

```
(pos (app p (app a)))
```

The inlined `zero-only/1` body descends schematically to:

```
(forall
  (tie y
    (or
      (neq (var x) (var y))
      (eq (var y) (app zero)))))
```

The public query API then probes the kernel in two directions:

```
(query/query-succeeds program query n fuel)
(query/query-fails    program query n fuel)
```

`query-succeeds` asks the kernel to close the negated query; `query-fails` asks it to close the query itself. Open-answer examples use `pf/run`, which binds visible answer variables in the style of miniKanren's `run`, translates the query to the same backend formula, and exports bindings from the proof-backed answer layer.

### 2.3 Fitting's Reference Programs

Fitting's first reference program defines even and odd natural numbers over the language with constant `zero` and unary successor `s`:

```
even(x) :- x = zero
        or exists y. (x = s(y) and odd(y)).

odd(x) :- forall y. (even(y) -> x != y).
```

The implemented frontend form is:

```
(pf/proflog p1-language
  (|- (even x)
    (or (= x zero)
        (exists [y]
          (and (= x (s y))
               (odd y)))))
  (|- (odd x)
    (forall [y]
      (implies (even y)
               (!= x y)))))
```

The second reference program is a one-or-two-token Nim game. A position wins when there is a legal move to a position that does not win:

```
win(x) :- exists y.
            ((x = s(y) or x = s(s(y))) and not win(y)).
```

The implementation keeps the move test inlined, as in Fitting's presentation:

```
(pf/proflog p2-language
  (|- (win x)
    (exists [y]
      (and (or (= x (s y))
               (= x (s (s y))))
           (not (win y))))))
```

Table 1 summarizes the focused Fitting suite results. The suite was run as a kernel-level test gate and completed with six tests, eighty-one assertions, and no failures or errors. It covers forward success, forward refutation, answer synthesis, partial synthesis, and unresolved classification.

The factored Nim case is included because it demonstrates that the evaluator is not merely computing the Nim recurrence in the host language. If the program introduces an auxiliary relation

```
win(x) :- exists y. (move(x, y) and not win(y)).
move(x, y) :- x = s(y) or x = s(s(y)).
```

then `move/2` itself is decidable on ground calls, but `win(1)` remains unresolved in the tested kernel setting. This is the operational boundary Fitting warned about: a logically tempting refactoring changes how procedure calls are admitted and discharged.

Table 1. Representative Fitting-program outcomes

| Case | Mode | Outcome |
|---|---|---|
| `even(0)` | forward success | succeeds |
| `odd(s(0))` | forward success | succeeds |
| `odd(0)` | forward refutation | fails |
| `win(4)` | forward success | succeeds |
| `win(3)` | forward refutation | fails |
| `move(1,0)` | auxiliary forward success | succeeds |
| `move(0,1)` | auxiliary forward refutation | fails |
| factored `win(1)` | status classification | unresolved |
| `append(x,y,[a,b,c])` | answer synthesis | target found |
| `reverse([a,b,c,a],cons(a,r))` | partial synthesis | target found |

### 2.4 Test Strategy

The implementation separates routine regression from slow semantic probes. The Fitting suite disables older host-side overlays while running, so its promoted rows cannot pass by fixture-specific evaluation. Representative proof objects are inspected for ordinary kernel and answer-overlay evidence, including procedure-call tags and the absence of diagnostic sidecar settlement tags. The same suite also includes finite-domain examples, list programs, and group verifier formulae to check that the proof kernel and proof profiles are not tailored only to the two paper examples.

## 3 DISCUSSION

Proflog is useful to the miniKanren setting because it gives another concrete point in the space between logic and computation. The program text is not a relational interpreter written in a host language, nor is it a Prolog program embedded in Clojure. It is a collection of first-order formulae whose operational content comes from tableau closure plus a procedure-call rule. That makes proof search, program evaluation, and proof-object construction part of one relational computation.

The implementation is also a cautionary example. Fitting's semantics are not Prolog's semantics with a different syntax. Query failure is proved, not assumed by search exhaustion. Undefined and unresolved cases are first-class outcomes. Refactorings that would be harmless in a purely extensional reading can change operational behavior when they introduce auxiliary procedure calls. These facts show up in tests rather than only in prose.

The current system is not presented as an efficient logic programming language. Some deeper recursive examples remain search-control frontiers, and several successful answer and partial-synthesis tests are intentionally kept out of the default fast path. The contribution is instead the executable correspondence: Fitting's theoretical Proflog can be mechanized as a proof-producing miniKanren-style kernel, and its non-trivial examples can be evaluated through that kernel with correctness results visible at the proof boundary.

## REFERENCES

[1] Nada Amin. 2026. leanTAP: A Theorem Prover for First-Order Classical Logic. https://github.com/namin/leanTAP. Clojure/core.logic.nominal implementation reference for leanTAP and alphaleanTAP. Accessed 2026-05-18.

[2] William E. Byrd. 2009. *Relational Programming in miniKanren: Techniques, Applications, and Implementations.* Ph. D. Dissertation. Indiana University. https://hdl.handle.net/2022/8777

[3] Melvin Fitting. 1994. Tableaux for Logic Programming. *Journal of Automated Reasoning* 13, 2 (1994), 175–188. https://melvinfitting.org/bookspapers/pdf/papers/LPTableaus.pdf

[4] Joseph P. Near, William E. Byrd, and Daniel P. Friedman. 2008. alphaleanTAP: A Declarative Theorem Prover for First-Order Classical Logic. In *Logic Programming: Proceedings of the 24th International Conference on Logic Programming (Lecture Notes in Computer Science, Vol. 5366)*. Springer, Udine, Italy, 238–252. https://people.csail.mit.edu/jnear/papers/alphatap.pdf

[5] David Nolen and contributors. 2026. core.logic. https://github.com/clojure/core.logic. Clojure logic programming library. Accessed 2026-05-18.